

データマイニング特論

第4回

データの preprocessing と特徴量エンジニアリング

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの事前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1

データクリーニング

データ解析をはじめる前に(その1)



なぜ前処理が必要か

>>> Garbage In, Garbage Out (GIGO)

データの品質問題に気付かずモデルを構築すると、結果は無意味になる。

⇒機械学習モデルの性能は「アルゴリズムの選択」よりも「データの質」に左右される

- ・欠損・外れ値：モデルはNaNを処理できない。外れ値は推定を歪める。
 - ⇒scikit-learn のほとんどのモデルは NaN をそのまま受け付けない（エラーになる）。
 - ⇒外れ値が1点あるだけで、平均・分散・相関係数がすべて変わってしまう。
 - ⇒例：身長データに 1700cm（入力ミス）が1件あるだけで平均が大きく狂う。
- ・スケールの不一致：距離ベースの手法はスケールに敏感。k-NN / SVM / PCA が影響を受ける。
 - ⇒第3回EDAで確認した proline (278~1680) vs alcohol (11~15) の問題。
 - ⇒k-NNは「距離」でデータを比較するため、大きい値の列が計算を支配してしまう。
 - ⇒例：proline の差が1000でも alcohol の差が1でも、proline の方が「遠い」と判断される。

※ k-NN：k近傍法、 SVM：サポートベクターマシン、 PCA：主成分分析

- ・カテゴリ変数：文字列はそのまま使えない。数値への変換が必要。
 - ⇒「Piedmont（ピエモンテ）」「Tuscany（トスカーナ）」という文字列は数値演算できない。
 - ⇒ただし、単純に 0, 1, 2 と置き換えると「 $0 < 1 < 2$ 」という偽の順序関係ができる。

本日の環境準備 (I)

いろいろ試してみよう！

日本語対応のモジュールをインストールする(その1)

```
!pip install japanize-matplotlib
```

日本語対応のモジュールをインストールする(その2)

```
!apt-get install -y fonts-ipafont-gothic
```

本日の環境準備 (2)

いろいろ試してみよう！

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import japanize_matplotlib
import seaborn as sns
import warnings
from sklearn.datasets import load_wine

warnings.filterwarnings('ignore')
np.random.seed(42)
sns.set_theme(style='whitegrid', palette='colorblind')
plt.rcParams['figure.dpi'] = 100

mpl.rcParams['font.family'] = 'IPAPGothic'
plt.rcParams['font.family'] = 'IPAexGothic'

# ===== Wine データの読み込み =====
wine = load_wine()
df_base = pd.DataFrame(wine.data, columns=wine.feature_names)
df_base['target'] = wine.target
df_base['target_name'] = df_base['target'].map({0: 'class_0', 1: 'class_1', 2: 'class_2'})

print(f'データセット: {df_base.shape[0]} 行 × {df_base.shape[1]} 列')
print(f'特徴量: {wine.feature_names}')
df_base.describe().T.round(2)
```

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several white circular and semi-circular lines of varying thicknesses. A prominent circular scale is visible on the left side, with numerical markings from 140 to 260 in increments of 10. Other circular elements include dashed lines, solid arcs, and small arrowheads pointing in different directions, creating a technical or scientific aesthetic.

欠損値の処理

欠損値の処理（Ⅰ）

>>> MCAR / MAR / MNARの判断が戦略の出発点

分類	意味	対処法の方向性
MCAR (Missing Completely At Random)	欠損は完全にランダム	リストワイズ削除でもバイアスは小さい
MAR (Missing At Random)	他の観測変数に依存して欠損	多重代入法が有効
MNAR (Missing Not At Random)	欠損値自体に依存	モデルベースの対処が必要

① 削除：【listwise】行ごと削除。【pairwise】列ごと削除。

（適用）欠損率 < 5% かつ MCAR

② 補完 (Imputation)：【定数補完】平均値・中央値・最頻値。【KNN補完】近傍サンプルの値を参照。
【反復補完】他列から回帰予測。

（適用）欠損率 5～30%

③ 欠損フラグの活用：補完 + フラグ列を追加。
→ 「欠損していたこと」自体がモデルへの情報になる。

（適用）欠損が MNAR の疑いがある時

欠損値の処理 (2)

いろいろ試してみよう！

===== Step 1: 欠損値の挿入=====

df = df_base.copy()

パターン1: ランダム欠損(MCAR)– alcohol 列

idx_mcar = np.random.choice(len(df), 20, replace=False)

df.loc[idx_mcar, 'alcohol'] = np.nan

パターン2: 条件付き欠損(MAR)– proline が低い時に magnesium が欠損しやすい

low_proline = df[df['proline'] < df['proline'].median()].index

idx_mar = np.random.choice(low_proline, 25, replace=False)

df.loc[idx_mar, 'magnesium'] = np.nan

パターン3: 共起欠損 – ash と alkalinity_of_ash が同時に欠損

idx_co = np.random.choice(len(df), 12, replace=False)

df.loc[idx_co, 'ash'] = np.nan

df.loc[idx_co, 'alkalinity_of_ash'] = np.nan

欠損率の確認

missing_rate = (df.isnull().sum() / len(df) * 100).round(1)

print('欠損率 (%):')

print(missing_rate[missing_rate > 0].to_string())

欠損率 (%):

alcohol 11.2

ash 6.7

alkalinity_of_ash 6.7

magnesium 14.0

欠損値の処理 (3)

いろいろ試してみよう！

===== Step 2: 欠損パターンの可視化 =====

try:

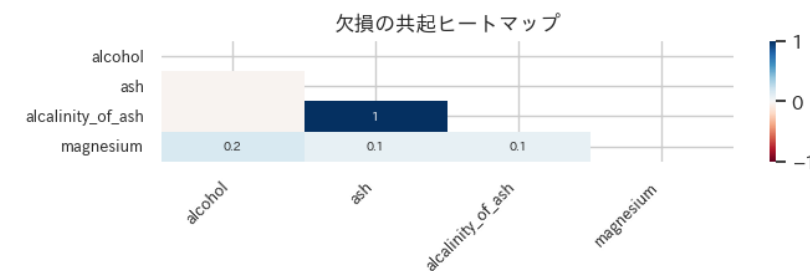
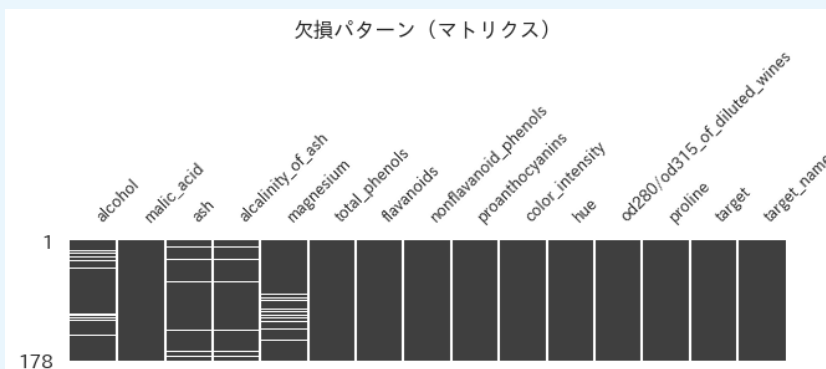
```
import missingno as msno
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
msno.matrix(df, ax=axes[0], fontsize=9)
axes[0].set_title('欠損パターン(マトリクス)')
msno.heatmap(df, ax=axes[1], fontsize=9)
axes[1].set_title('欠損の共起ヒートマップ')
plt.tight_layout()
plt.show()
```

except ImportError:

```
print('missingno が未インストールです。pip install missingno で導入できます。')
```

代替: 欠損マップをヒートマップで表示

```
fig, ax = plt.subplots(figsize=(12, 3))
sns.heatmap(df.isnull().T, cbar=False, cmap='Blues', ax=ax)
ax.set_title('欠損マップ(青 = 欠損)')
plt.tight_layout()
plt.show()
```



欠損値の処理 (4)

いろいろ試してみよう！

===== Step 3: 欠損値の補完 =====

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.model_selection import train_test_split
```

```
feature_cols = wine.feature_names
X = df[feature_cols]
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

--- ① 中央値補完(最もシンプル)---

```
imp_median = SimpleImputer(strategy='median')
X_train_med = pd.DataFrame(imp_median.fit_transform(X_train), columns=feature_cols)
X_test_med = pd.DataFrame(imp_median.transform(X_test), columns=feature_cols)
```

--- ② KNN補完(近傍サンプルから推定)---

```
imp_knn = KNNImputer(n_neighbors=5)
X_train_knn = pd.DataFrame(imp_knn.fit_transform(X_train), columns=feature_cols)
X_test_knn = pd.DataFrame(imp_knn.transform(X_test), columns=feature_cols)
```

```
print('補完前の欠損数 (train):', X_train.isnull().sum().sum())
print('中央値補完後の欠損数:', X_train_med.isnull().sum().sum())
print('KNN補完後の欠損数:', X_train_knn.isnull().sum().sum())
```

補完前の欠損数 (train): 58

中央値補完後の欠損数: 0

KNN補完後の欠損数: 0

欠損値の処理 (5)

いろいろ試してみよう！

===== Step 4: 欠損フラグ列の追加 =====

from sklearn.impute import MissingIndicator

indicator = MissingIndicator(features='missing-only')

flags_train = indicator.fit_transform(X_train)

flags_test = indicator.transform(X_test)

欠損があった列名をフラグ列名に使用

flag_cols = [f'{feature_cols[i]}_missing'

for i in indicator.features_]

print('フラグ列:', flag_cols)

中央値補完 + フラグ を結合

X_train_with_flag = pd.concat([

X_train_med,

pd.DataFrame(flags_train.astype(int),

columns=flag_cols,

index=X_train_med.index)

], axis=1)

print(f'特徴量数: {X_train.shape[1]} → {X_train_with_flag.shape[1]} (フラグ追加後)')

フラグ列: ['alcohol_missing', 'ash_missing', 'alkalinity_of_ash_missing', 'magnesium_missing']

特徴量数: 13 → 17 (フラグ追加後)

欠損値の処理 (6)

いろいろ試してみよう！

===== Step 5: 補完手法の比較(alcohol 列)=====

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

for ax, data, title in zip(

axes,

[X_train.dropna(), X_train_med, X_train_knn],

['元データ(欠損除外)', '中央値補完', 'KNN補完']

):

sns.histplot(data['alcohol'], bins=20, kde=True, ax=ax, color='steelblue')

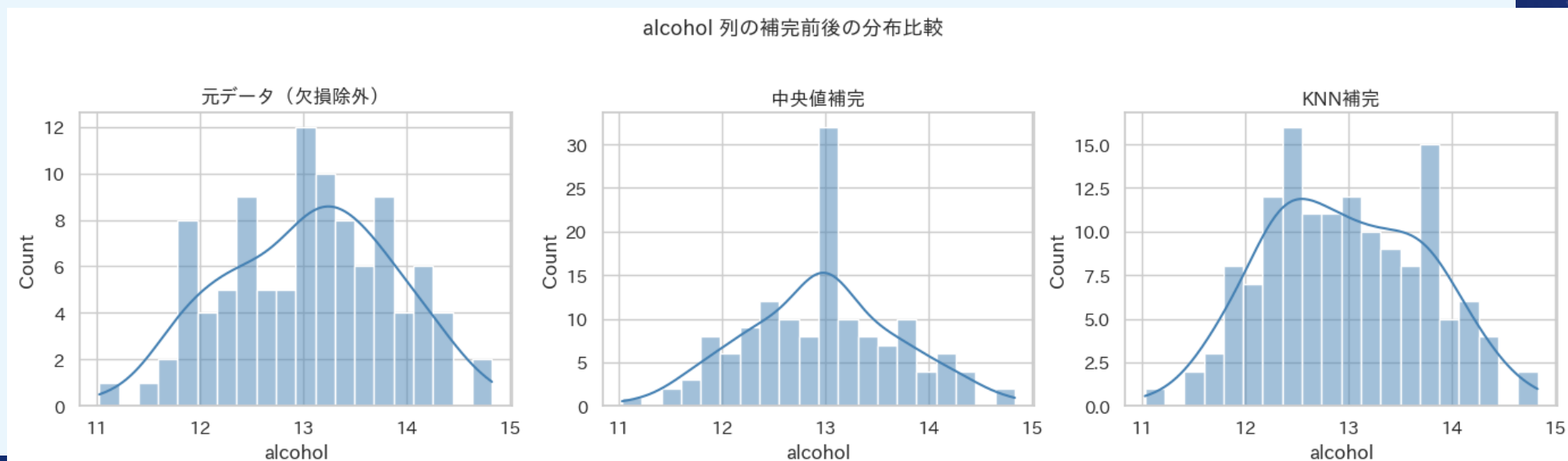
ax.set_title(title)

ax.set_xlabel('alcohol')

plt.suptitle('alcohol 列の補完前後の分布比較', y=1.02, fontsize=13)

plt.tight_layout()

plt.show()



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several white circular elements. A large circular scale with degree markings from 140 to 260 is prominent on the left side. Other smaller circles, some with arrows indicating rotation, are scattered across the image. The title text is centered in the middle of the frame.

外れ値の処理

外れ値の処理（Ⅰ）

>>> 検出方法

- ① IQR法： $Q3 + 1.5 \times IQR$ を超える点を外れ値とみなす（箱ひげ図の上下ひげ）
- ② zスコア法：標準化後に $|z| > 3$ の点を外れ値とみなす（正規分布の仮定あり）

>>> 処理方法

- ① 削除：外れ値の行を除去。N数が減るのに注意。
- ② クリッピング：上下限に丸める。情報は残る。
- ③ 対数変換：右裾を圧縮して正規分布に近づける。
- ④ そのまま：ドメイン知識で「意味のある外れ値」と判断。

注意：外れ値を消す = データを捏造するリスク。何をなぜ処理したか必ず記録する！

外れ値の処理 (2)

いろいろ試してみよう！

===== Step 1: 外れ値の確認(第3回EDAの復習) =====

```
df = df_base.copy()
feature_cols = wine.feature_names
```

意図的に外れ値を挿入(測定エラーを模擬)

```
df.loc[5, 'proline'] = 2800.0 # 正常範囲の約2倍
```

```
df.loc[42, 'proline'] = 3100.0
```

```
df.loc[10, 'alcohol'] = 22.0 # ワインとして有り得ない値
```

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
```

```
for ax, col in zip(axes, ['proline', 'alcohol', 'flavanoids']):
```

```
    sns.boxplot(x='target_name', y=col, data=df, palette='Set2', ax=ax, width=0.5)
```

```
    sns.stripplot(x='target_name', y=col, data=df, color='black', alpha=0.3, size=3, ax=ax)
```

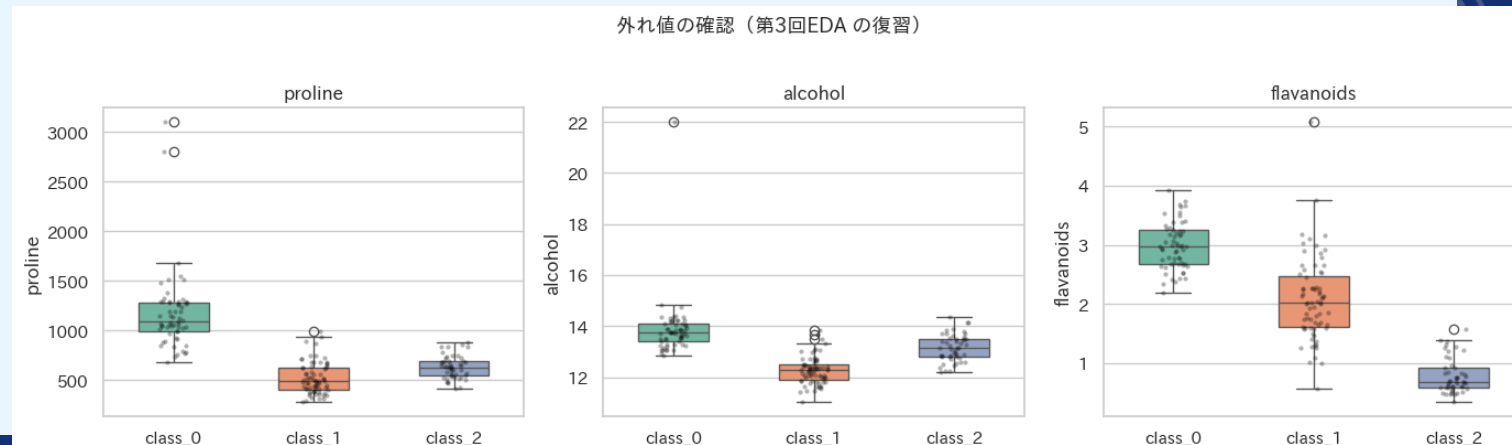
```
    ax.set_title(col)
```

```
    ax.set_xlabel('')
```

```
plt.suptitle('外れ値の確認(第3回EDA の復習)', y=1.02, fontsize=13)
```

```
plt.tight_layout()
```

```
plt.show()
```



外れ値の処理 (3)

いろいろ試してみよう！

===== Step 2: IQR 法による外れ値の検出 =====

```
def detect_outliers_iqr(series, k=1.5):  
    Q1 = series.quantile(0.25)  
    Q3 = series.quantile(0.75)  
    IQR = Q3 - Q1  
    lower = Q1 - k * IQR  
    upper = Q3 + k * IQR  
    mask = (series < lower) | (series > upper)  
    return mask, lower, upper  
  
for col in ['proline', 'alcohol', 'flavanoids']:  
    mask, lo, hi = detect_outliers_iqr(df[col])  
    n = mask.sum()  
    print(f'{col:30s}: 外れ値 {n:3d} 件 (下限={lo:.1f}, 上限={hi:.1f})')
```

```
proline           : 外れ値  2 件 (下限=-226.2, 上限=1711.8)  
alcohol           : 外れ値  1 件 (下限=10.4, 上限=15.7)  
flavanoids        : 外れ値  0 件 (下限=-1.3, 上限=5.4)
```

外れ値の処理（４）

いろいろ試してみよう！

===== Step 3: Zスコア法による外れ値の検出 =====

from scipy import stats

z_scores = np.abs(stats.zscore(df[feature_cols].fillna(df[feature_cols].median())))

outlier_mask_z = (z_scores > 3).any(axis=1)

print(f'Zスコア法（ $|z|>3$ ）で検出された外れ値サンプル数: {outlier_mask_z.sum()}')

print('該当インデックス:', df[outlier_mask_z].index.tolist())

Zスコア法（ $|z|>3$ ）で検出された外れ値サンプル数: 13

該当インデックス: [5, 10, 25, 42, 59, 69, 73, 95, 110, 115, 121, 123, 158]

外れ値の処理 (5)

いろいろ試してみよう！

===== Step 4: 処理方法の比較 =====

df_proc = df.copy()

① クリッピング(上下限に丸める)

mask_p, lo_p, hi_p = detect_outliers_iqr(df_proc['proline'])

df_proc['proline_clipped'] = df_proc['proline'].clip(lower=lo_p, upper=hi_p)

② 対数変換(正の値のみ適用可)

df_proc['proline_log'] = np.log1p(df_proc['proline'])

比較プロット

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

```
for ax, (col, title) in zip(axes, [ ('proline', '元データ(外れ値あり)'), ('proline_clipped', 'クリッピング後'),
                                   ('proline_log', '対数変換後')]):
    sns.histplot(df_proc[col], bins=25, kde=True, ax=ax, color='steelblue')
    ax.set_title(title)
    ax.set_xlabel(col)
```

plt.suptitle('proline の外れ値処理比較', y=1.02, fontsize=13)

plt.tight_layout()

plt.show()

```
print('元データ: 平均={:.1f}, 標準偏差={:.1f}, 最大={:.1f}'.format(
    df['proline'].mean(), df['proline'].std(), df['proline'].max()))
```

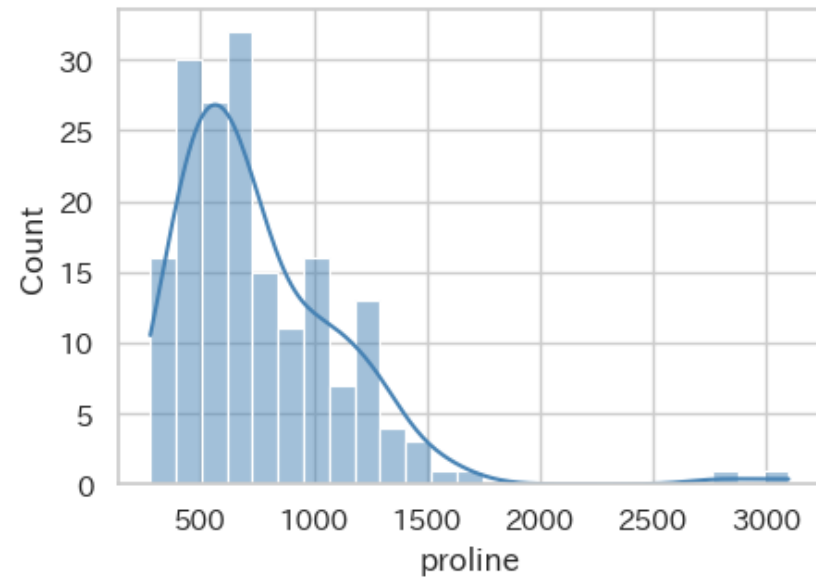
```
print('クリップ後: 平均={:.1f}, 標準偏差={:.1f}, 最大={:.1f}'.format(df_proc['proline_clipped'].mean(),
    df_proc['proline_clipped'].std(), df_proc['proline_clipped'].max()))
```

データの前処理と特徴量エンジニアリング

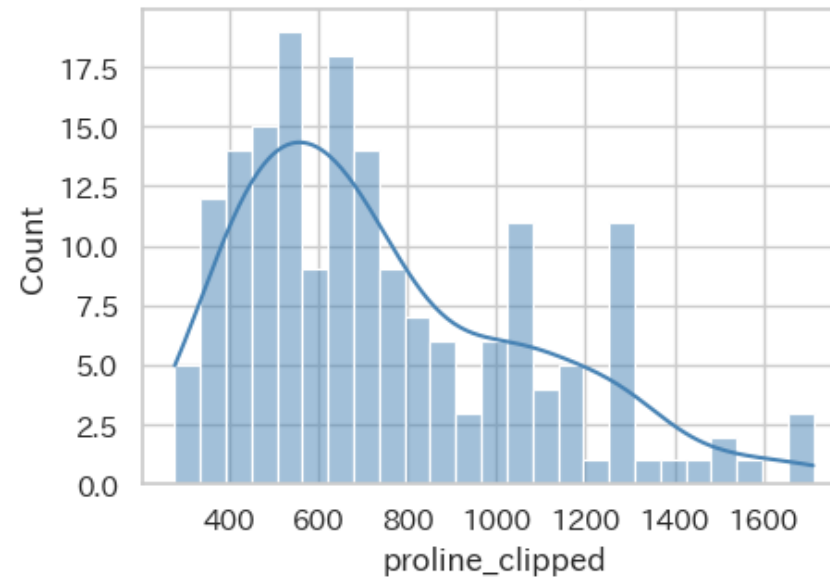
外れ値の処理（6）

proline の外れ値処理比較

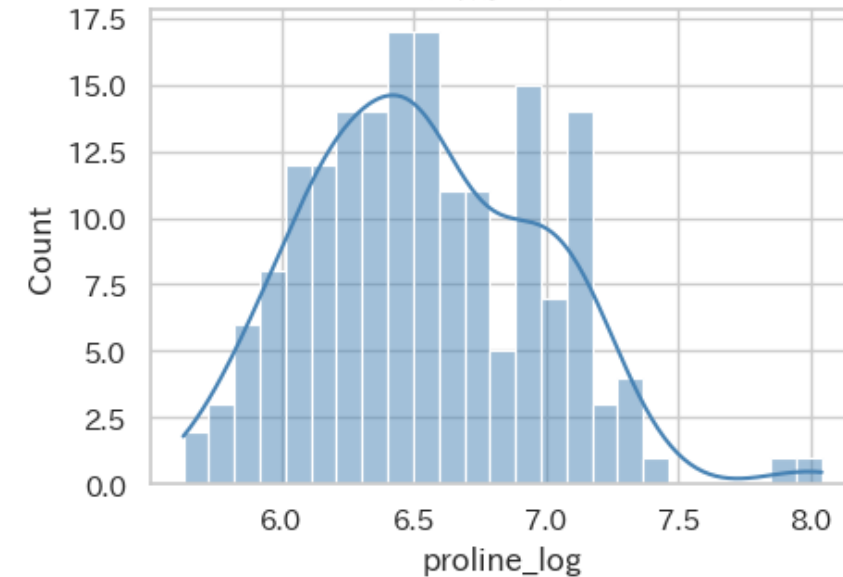
元データ（外れ値あり）



クリッピング後



対数変換後



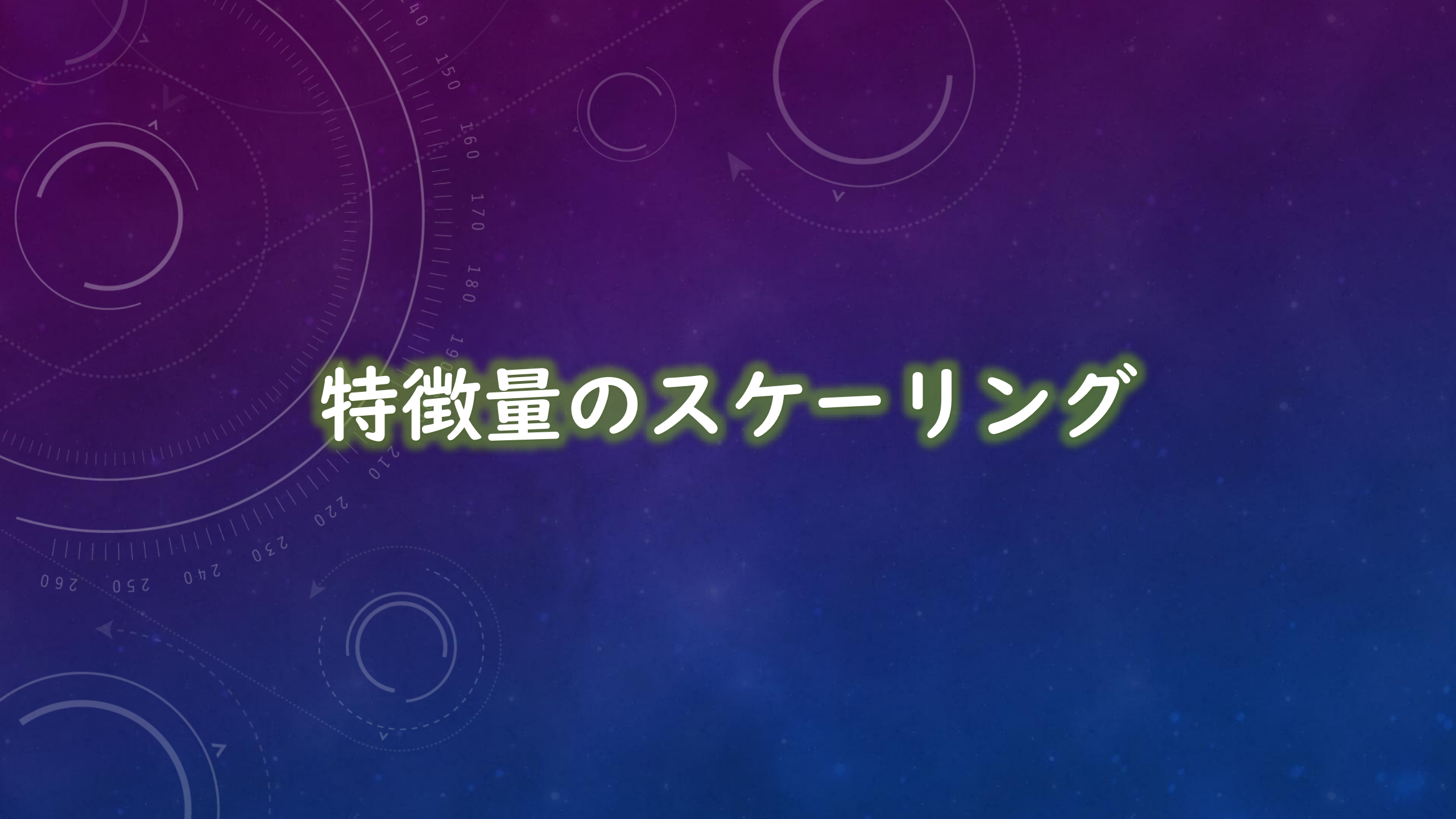
元データ: 平均=765.7, 標準偏差=387.9, 最大=3100.0
クリップ後: 平均=751.8, 標準偏差=325.9, 最大=1711.8

2

データの変換

データ解析を始める前に(その2)



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale is visible on the left side, with numerical markings from 140 to 260 in increments of 10. Other smaller circles and arcs are scattered across the frame, some with arrows indicating a clockwise direction.

特徴量のスケールリング

特徴量のスケールリング (1)

>>> 第3回Wineデータの確認 —— prolineだけスケールが桁違い！

proline: 278~1680 vs alcohol: 11.0~14.8 → 距離計算が proline に支配される

>>> 主な方法

- ① 標準化 (Standard Scaler) : $z = (x - \mu) / \sigma$
 - ⇒ 平均0, 標準偏差1に変換。 (正規分布を仮定)
 - ⇒ 外れ値の影響を受ける。
- ② 正規化 (Min-Max Scaler) : $x' = (x - \min) / (\max - \min)$
 - ⇒ 0~1の範囲に変換。 (ニューラルネットに適する)
 - ⇒ 外れ値に弱い。
- ③ ロバスト (Robust Scaler) 対数変換 : $x' = (x - Q_2) / IQR$
 - ⇒ 中央値とIQRを使用。 (外れ値に強い)
 - ⇒ スケールが直感的でない

特徴量のスケーリング (2)

いろいろ試してみよう！

===== Step 1: スケール問題の可視化(第3回EDAの復習) =====

```
df = df_base.copy()
feature_cols = wine.feature_names
X = df[feature_cols]

print('各列の値の範囲:')
summary = pd.DataFrame({'min': X.min(), 'max': X.max(), 'range': X.max() - X.min()
                        }).sort_values('range', ascending=False)
print(summary.round(2).to_string())
```

各列の値の範囲:

	min	max	range
proline	278.00	1680.00	1402.00
magnesium	70.00	162.00	92.00
alcalinity_of_ash	10.60	30.00	19.40
color_intensity	1.28	13.00	11.72
malic_acid	0.74	5.80	5.06
flavanoids	0.34	5.08	4.74
alcohol	11.03	14.83	3.80
proanthocyanins	0.41	3.58	3.17
total_phenols	0.98	3.88	2.90
od280/od315_of_diluted_wines	1.27	4.00	2.73
ash	1.36	3.23	1.87
hue	0.48	1.71	1.23
nonflavanoid_phenols	0.13	0.66	0.53

特徴量のスケーリング (3)

いろいろ試してみよう！

===== Step 2: 各スケーラーの適用と比較 =====

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.model_selection import train_test_split
```

```
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
scalers = {
    'StandardScaler': StandardScaler(),
    'MinMaxScaler':   MinMaxScaler(),
    'RobustScaler':   RobustScaler(),
}
```

```
scaled = {}
for name, scaler in scalers.items():
    X_tr = pd.DataFrame(
        scaler.fit_transform(X_train), columns=feature_cols)
    X_te = pd.DataFrame(
        scaler.transform(X_test), columns=feature_cols)
    scaled[name] = (X_tr, X_te)
```


特徴量のスケーリング (4)

いろいろ試してみよう！

proline と alcohol の変換前後を比較

fig, axes = plt.subplots(2, 4, figsize=(16, 7))

for row, col in enumerate(['proline', 'alcohol']):

元データ

sns.boxplot(y=X_train[col], ax=axes[row, 0], color='lightgray')

axes[row, 0].set_title('元データ')

axes[row, 0].set_xlabel(col)

for i, (name, (X_tr, _)) in enumerate(scaled.items()):

sns.boxplot(y=X_tr[col], ax=axes[row, i+1], color=['steelblue', 'coral', 'seagreen'][i])

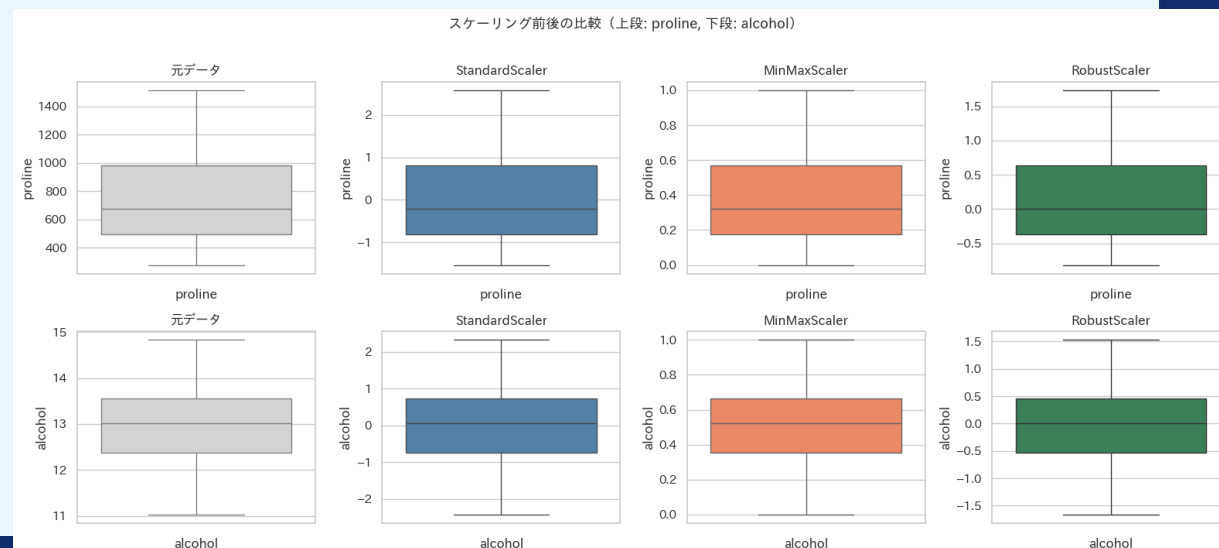
axes[row, i+1].set_title(name)

axes[row, i+1].set_xlabel(col)

plt.suptitle('スケーリング前後の比較(上段: proline, 下段: alcohol)', y=1.01, fontsize=13)

plt.tight_layout()

plt.show()



===== Step 3: スケーリングがモデルに与える影響(k-NN の例) =====

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
results = {}
```

スケーリングなし

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
results['スケーリングなし'] = accuracy_score(y_test, knn.predict(X_test))
```

各スケーラー

```
for name, (X_tr, X_te) in scaled.items():
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_tr, y_train)
    results[name] = accuracy_score(y_test, knn.predict(X_te))
```

```
print('k-NN (k=5) の精度比較:')
for name, acc in results.items():
    bar = '■' * int(acc * 30)
    print(f' {name:20s}: {acc:.3f} {bar}')
```

k-NN (k=5) の精度比較:

スケールなし : 0.806

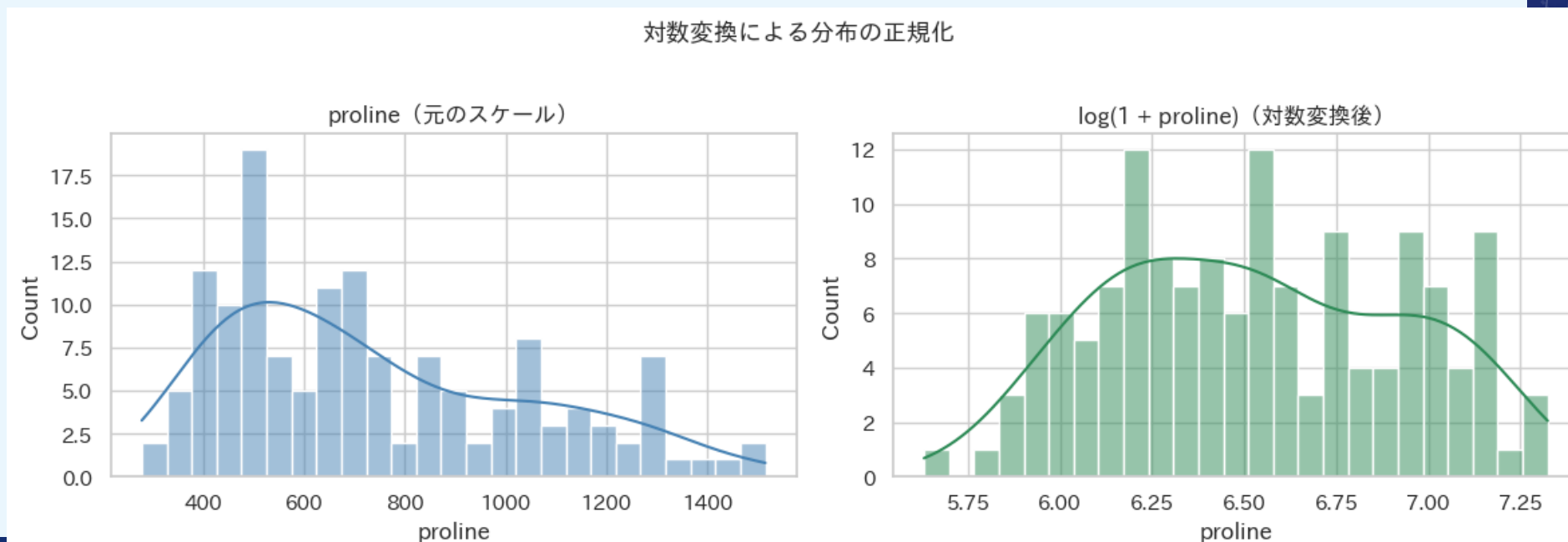
StandardScaler	MinMaxScaler
0.92	1.000

RobustScaler	: 0.972	
--------------	---------	--

特徴量のスケーリング (6)

いろいろ試してみよう！

```
# ===== Step 4: 対数変換(proline の右裾を圧縮) =====  
fig, axes = plt.subplots(1, 2, figsize=(12, 4))  
  
sns.histplot(X_train['proline'], bins=25, kde=True,  
             ax=axes[0], color='steelblue')  
axes[0].set_title('proline(元のスケール)')  
  
sns.histplot(np.log1p(X_train['proline']), bins=25, kde=True,  
             ax=axes[1], color='seagreen')  
axes[1].set_title('log(1 + proline)(対数変換後)')  
  
plt.suptitle('対数変換による分布の正規化', y=1.02, fontsize=13)  
plt.tight_layout()  
plt.show()
```



カテゴリ変換のエンコーディング

カテゴリ変換のエンコーディング（１）

>>> 主な方法

- ① ラベルエンコーディング (Label Encoding) : 順序がある変数 (小・中・大 → 0・1・2)
⇒ 順序がない変数に使うと「 $0 < 1 < 2$ 」という偽の大小関係をモデルが学習してしまう
- ② ダミー変数化 (One-Hot Encoding) : 順序がないカテゴリ変数 (赤・青・緑 → 3列の0/1)
⇒ 値の種類が多い場合 (100種類以上) は次元爆発
⇒ 別の手法を検討
- ③ ターゲットエンコーディング (Target Encoding) : 高カーディナリティ変数 (都道府県、品種名など)
⇒ データリークageジのリスク！
⇒ 交差検証との組み合わせが必須

カテゴリ変換のエンコーディング (2)

いろいろ試してみよう！

===== Step 1: Wineデータにカテゴリ列を追加 =====

df = df_base.copy()

品質ランク(順序あり): alcohol の値から3段階に分類

df['quality_rank'] = pd.cut(df['alcohol'], bins=[0, 12.5, 13.5, 100], labels=['low', 'medium', 'high'])

産地(順序なし): ランダムに割り当て

np.random.seed(42)

df['region'] = np.random.choice(['Piedmont', 'Tuscany', 'Veneto'], len(df))

print('カテゴリ列の確認:')

print(df[['quality_rank', 'region', 'target_name']].head(10))

print('\nquality_rank の分布:')

print(df['quality_rank'].value_counts())

カテゴリ列の確認:

	quality_rank	region	target_name
0	high	Veneto	class_0
1	medium	Piedmont	class_0
2	medium	Veneto	class_0
3	high	Veneto	class_0
4	medium	Piedmont	class_0
5	high	Piedmont	class_0
6	high	Veneto	class_0
7	high	Tuscany	class_0
8	high	Veneto	class_0
9	high	Veneto	class_0

quality_rank の分布:

quality_rank	
medium	66
low	57
high	55

Name: count, dtype: int64

カテゴリ変換のエンコーディング (3)

いろいろ試してみよう！

```
# ===== Step 2: ラベルエンコーディング(順序あり: quality_rank) =====  
from sklearn.preprocessing import OrdinalEncoder
```

```
# 順序を明示的に指定
```

```
oe = OrdinalEncoder(categories=[['low', 'medium', 'high']])  
df['quality_rank_enc'] = oe.fit_transform(  
    df[['quality_rank']].astype(str)).astype(int)
```

```
print('エンコーディング結果(quality_rank):')  
print(df[['quality_rank', 'quality_rank_enc']  
       .drop_duplicates()  
       .sort_values('quality_rank_enc'))
```

```
エンコーディング結果(quality_rank):
```

	quality_rank	quality_rank_enc
59	low	0
1	medium	1
0	high	2

カテゴリ変換のエンコーディング（４）

いろいろ試してみよう！

===== Step 3: One-Hot エンコーディング(順序なし: region) =====

from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False, drop='first') # drop='first' で多重共線性を回避

region_enc = ohe.fit_transform(df[['region']])

region_df = pd.DataFrame(region_enc, columns=ohe.get_feature_names_out(['region']))

print('One-Hot エンコーディング結果(region):')

print(pd.concat([df[['region']], region_df], axis=1).drop_duplicates(subset='region'))

pandas による簡易版(確認用)

print('¥npd.get_dummies による結果:')

print(pd.get_dummies(df['region'], drop_first=True, dtype=int).head())

One-Hot エンコーディング結果(region):

	region	region_Tuscany	region_Veneto
0	Veneto	0.0	1.0
1	Piedmont	0.0	0.0
7	Tuscany	1.0	0.0

pd.get_dummies による結果:

	Tuscany	Veneto
0	0	1
1	0	0
2	0	1
3	0	1
4	0	0

カテゴリ変換のエンコーディング (5)

いろいろ試してみよう！

```
# ===== Step 4: エンコーディング前後のデータ比較 =====

# --- quality_rank: エンコーディング前後 ---
print('【OrdinalEncoder: quality_rank の変換前後】')
comparison_ord = df[['quality_rank', 'quality_rank_enc']].drop_duplicates()
print(comparison_ord.sort_values('quality_rank_enc'))

# --- region: One-Hot エンコーディング前後 ---
print('\n【OneHotEncoder: region の変換前後】')
region_before = df[['region']].head(8)
region_after = pd.DataFrame(
    ohe.transform(region_before),
    columns=ohe.get_feature_names_out(['region']))
comparison_ohe = pd.concat([region_before.reset_index(drop=True),
                           region_after], axis=1)

print(comparison_ohe)

# --- 可視化: 変換前の度数 vs 変換後の列の分布 ---
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 変換前: quality_rank の度数
df['quality_rank'].value_counts().reindex(['low', 'medium', 'high'])\
    .plot.bar(ax=axes[0], color=['steelblue', 'coral', 'seagreen'],
              edgecolor='white')
axes[0].set_title('変換前: quality_rank の度数')
axes[0].tick_params(axis='x', rotation=0)
```

カテゴリ変換のエンコーディング (6)

いろいろ試してみよう！

変換後: quality_rank_enc の分布

```
df['quality_rank_enc'].value_counts().sort_index()¥
    .plot.bar(ax=axes[1], color=['steelblue','coral','seagreen'], edgecolor='white')
axes[1].set_title('変換後: quality_rank_enc (0/1/2)')
axes[1].set_xlabel('エンコード値')
axes[1].tick_params(axis='x', rotation=0)
```

変換後: One-Hot 列の合計(region の分布が保存されているか確認)

```
region_after_full = pd.DataFrame(ohe.transform(df[['region']]), columns=ohe.get_feature_names_out(['region']))
region_after_full.sum().plot.bar(ax=axes[2], color=['steelblue','coral'], edgecolor='white')
axes[2].set_title('変換後: One-Hot 列の合計¥n(regionの分布が保存されているか)')
axes[2].tick_params(axis='x', rotation=15)
```

```
plt.suptitle('エンコーディング前後の比較', y=1.02, fontsize=13)
plt.tight_layout()
plt.show()
```

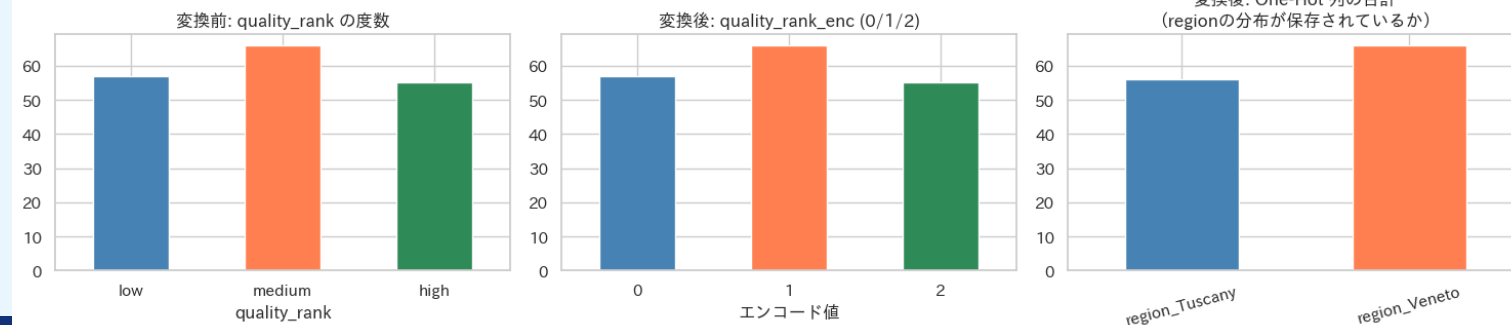
【OrdinalEncoder: quality_rank の変換前後】

quality_rank	quality_rank_enc
low	0
medium	1
high	2

【OneHotEncoder: region の変換前後】

region	region_Tuscany	region_Veneto
Veneto	0.0	1.0
Piedmont	0.0	0.0
Veneto	0.0	1.0
Veneto	0.0	1.0
Piedmont	0.0	0.0
Piedmont	0.0	0.0
Veneto	0.0	1.0
Tuscany	1.0	0.0

エンコーディング前後の比較

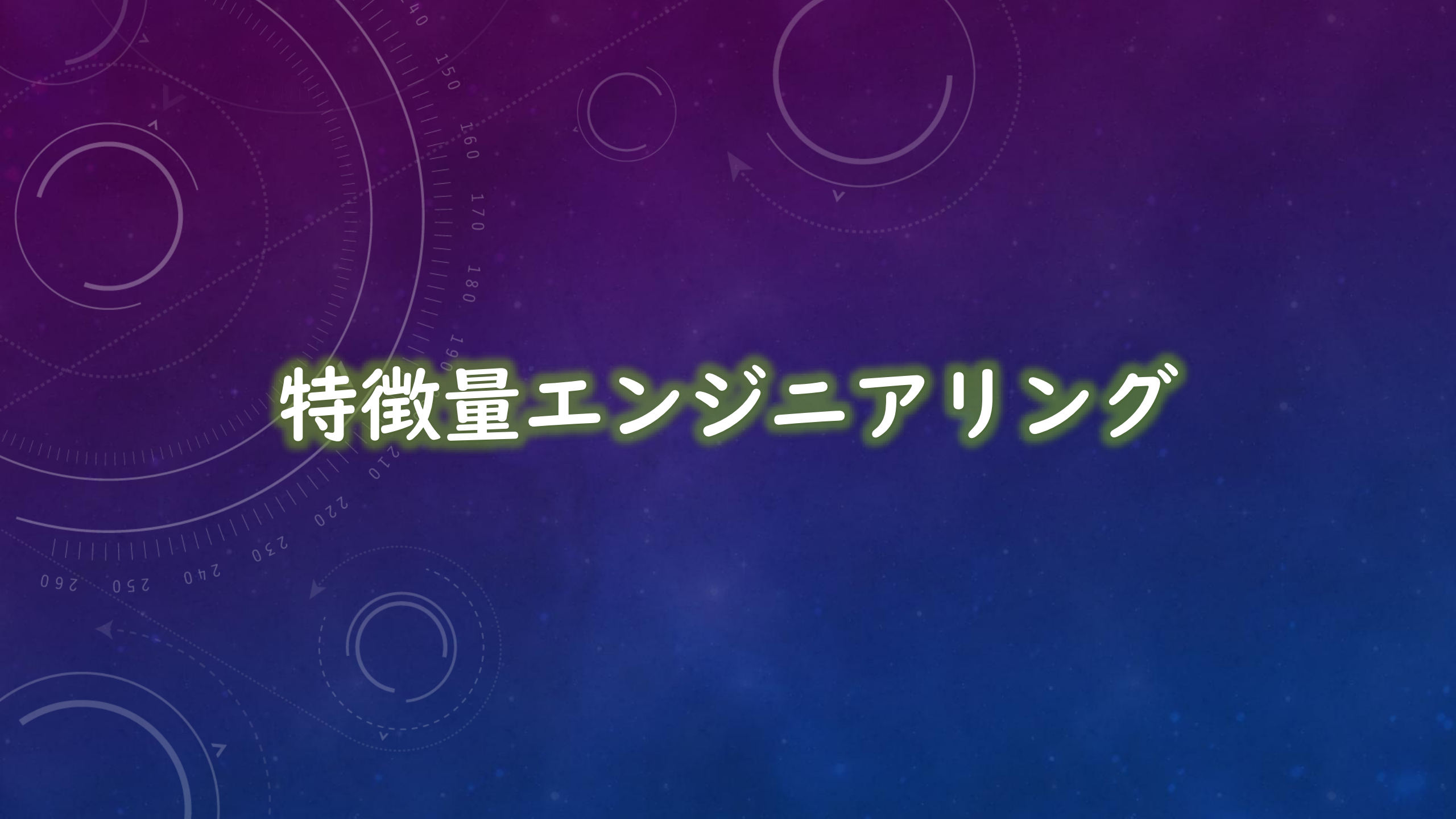


3

特徴量エンジニアリング

データ解析を始める前に(その3)



The background features a dark blue gradient with a subtle pattern of white stars and constellations. Overlaid on this are several technical diagrams in a lighter blue color. These include circular gauges with radial scales and tick marks, some with numerical labels like 150, 160, 170, 180, 190, 210, 220, 230, 240, 250, and 260. There are also circular arrows indicating clockwise or counter-clockwise rotation, and dashed lines representing paths or orbits.

特徴量エンジニアリング

特徴量エンジニアリング（I）

>>> ドメイン知識が最も活きる工程 — 「データに知恵を加える」

⇒ アルゴリズムを磨くより、特徴量を磨く方が性能改善に直結することが多い

>>> 特徴量の作成

- ⇒ 既存列の組み合わせ : BMI = 体重 / 身長²
- ⇒ 比率・差分 : seed_weight / pod_length (粒/cm)
- ⇒ 交互作用項 : $x_1 \times x_2$
- ⇒ 時系列 : 前日との差分、移動平均
- ⇒ 集約 : グループ別の平均・最大値

>>> 特徴量の選択

- ① フィルタ法 : 相関係数・分散によるスクリーニング
- ② ラッパー法 : RFE (再帰的特徴量除去)
- ③ 埋め込み法 : Lasso の係数 / RF の重要度

→ 次元の呪い / 過学習 / 計算コスト の削減

特徴量エンジニアリング (2)

いろいろ試してみよう！

===== Step 1: 特徴量の作成 =====

df = df_base.copy()

① 比率特徴量(化学的に意味のある組み合わせ)

df['flavanoids_per_alcohol'] = df['flavanoids'] / df['alcohol']

df['phenols_ratio'] = df['total_phenols'] / (df['nonflavanoid_phenols'] + 1e-6)

df['color_per_hue'] = df['color_intensity'] / (df['hue'] + 1e-6)

② 差分特徴量

df['phenols_diff'] = df['total_phenols'] - df['nonflavanoid_phenols']

③ 対数変換(右裾の長い特徴量)

df['proline_log'] = np.log1p(df['proline'])

④ 二乗特徴量(非線形関係の捕捉)

df['alcohol_sq'] = df['alcohol'] ** 2

```
new_features = [  
    'flavanoids_per_alcohol', 'phenols_ratio',  
    'color_per_hue', 'phenols_diff', 'proline_log', 'alcohol_sq'  
]
```

print('新規特徴量の統計量:')

df[new_features].describe().T.round(3)

新規特徴量の統計量:

	count	mean	std	min	25%	50%	75%	max
flavanoids_per_alcohol	178.0	0.156	0.075	0.024	0.092	0.172	0.211	0.439
phenols_ratio	178.0	7.473	4.080	2.379	4.082	6.994	9.598	26.000
color_per_hue	178.0	6.039	4.347	1.111	3.140	4.777	6.694	22.807
phenols_diff	178.0	1.933	0.691	0.580	1.282	1.975	2.465	3.610
proline_log	178.0	6.532	0.414	5.631	6.218	6.514	6.894	7.427
alcohol_sq	178.0	169.671	21.087	121.661	152.831	170.302	187.074	219.929

特徴量エンジニアリング (3)

いろいろ試してみよう！

===== Step 2: 新特徴量の識別力を確認 =====

fig, axes = plt.subplots(2, 3, figsize=(15, 8))

axes = axes.ravel()

for ax, feat in zip(axes, new_features):

sns.boxplot(x='target_name', y=feat, data=df, palette='Set2', width=0.5, ax=ax)

sns.stripplot(x='target_name', y=feat, data=df, color='black', alpha=0.25, size=3, ax=ax)

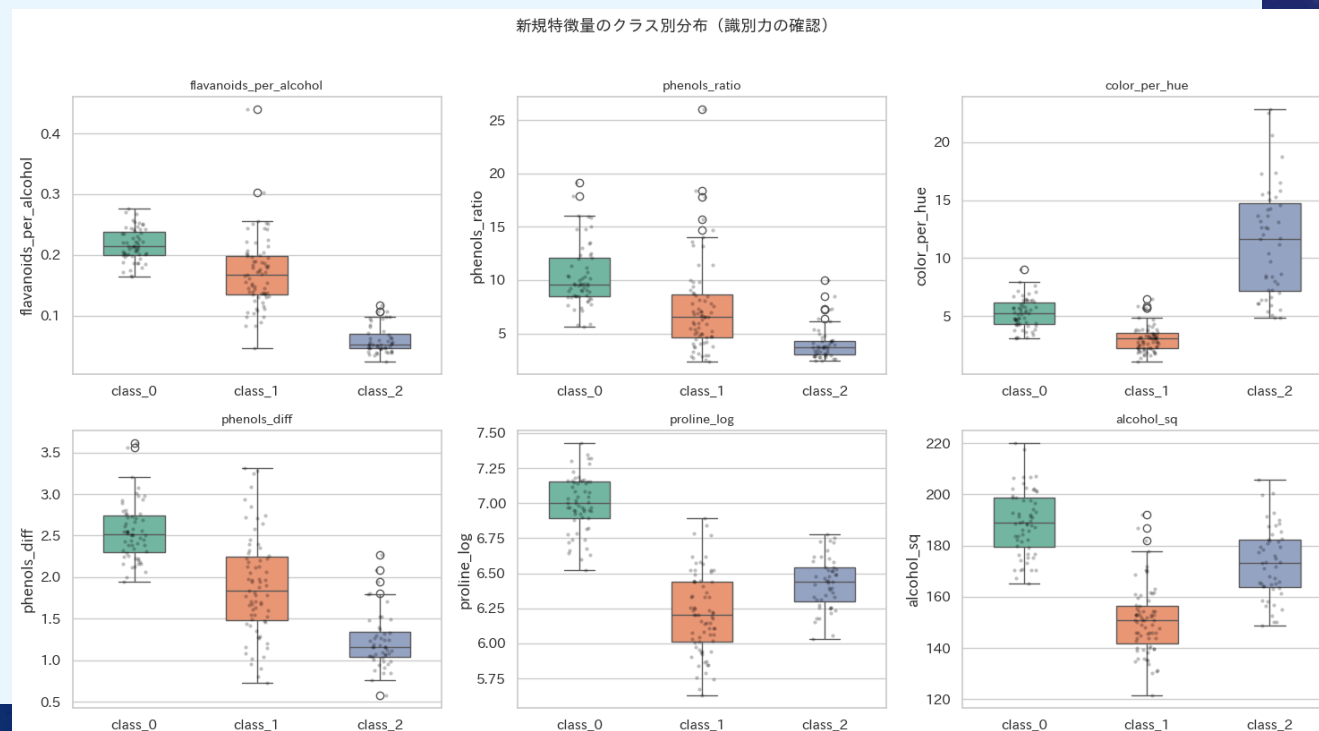
ax.set_title(feat, fontsize=10)

ax.set_xlabel('')

plt.suptitle('新規特徴量のクラス別分布(識別力の確認)', y=1.02, fontsize=13)

plt.tight_layout()

plt.show()



特徴量エンジニアリング（４）

いろいろ試してみよう！

===== Step 3: 特徴量の選択 =====

```
from sklearn.feature_selection import SelectKBest, f_classif, RFE, VarianceThreshold
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

全特徴量(元 + 新規)

```
all_features = wine.feature_names + new_features
X_all = df[all_features]
y      = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y, test_size=0.2, random_state=42, stratify=y)
```

標準化

```
scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_train)
X_te_sc = scaler.transform(X_test)
```

特徴量エンジニアリング (5)

いろいろ試してみよう！

① フィルタ法: ANOVA F統計量 による上位10特徴量

```
selector_kbest = SelectKBest(f_classif, k=10)
selector_kbest.fit(X_tr_sc, y_train)
selected_kbest = [all_features[i] for i in selector_kbest.get_support(indices=True)]
print('① SelectKBest (top 10):')
print(' ', selected_kbest)
```

② 埋め込み法: ランダムフォレストの特徴量重要度

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_tr_sc, y_train)
importances = pd.Series(rf.feature_importances_, index=all_features)
top10_rf = importances.sort_values(ascending=False).head(10)
print('¥n② RandomForest 特徴量重要度 (top 10):')
print(top10_rf.round(4).to_string())
```

① SelectKBest (top 10):

```
['alcohol', 'flavanoids', 'color_intensity', 'od280/od315_of_diluted_wines', 'proline',
'flavanoids_per_alcohol', 'color_per_hue', 'phenols_diff', 'proline_log', 'alcohol_sq']
```

② RandomForest 特徴量重要度 (top 10):

flavanoids_per_alcohol	0.1201
color_intensity	0.1174
color_per_hue	0.1146
flavanoids	0.1042
proline	0.0865
alcohol_sq	0.0766
proline_log	0.0766
od280/od315_of_diluted_wines	0.0665
alcohol	0.0524
hue	0.0477

特徴量エンジニアリング (6)

いろいろ試してみよう！

===== Step 4: 特徴量重要度の可視化 =====

fig, ax = plt.subplots(figsize=(10, 5))

top10_rf.sort_values().plot.barh(

ax=ax,

color=['coral' if f in new_features else 'steelblue'

for f in top10_rf.sort_values().index]

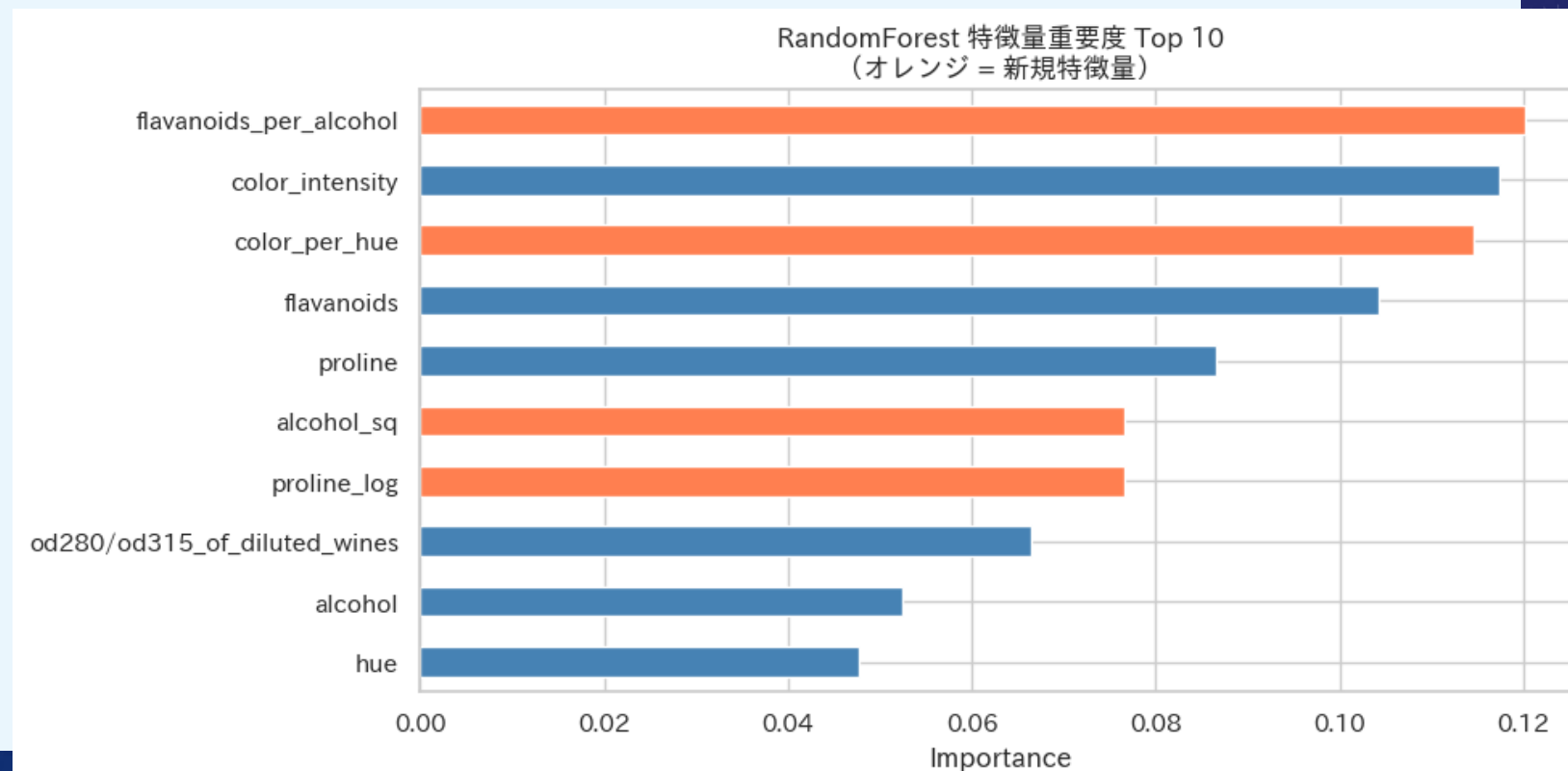
)

ax.set_title('RandomForest 特徴量重要度 Top 10¥n(オレンジ = 新規特徴量)', fontsize=12)

ax.set_xlabel('Importance')

plt.tight_layout()

plt.show()



4

本日の課題

頑張ってください。



課題

【課題】 期限：2026/5/13（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第4回（5/7）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力
!jupyter nbconvert --to html "aaa.ipynb"